FeedbackLens — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── samples/
│   └── reviews.txt
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── complaints.py
│   │   ├── praises.py
│   │   └── summary.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── pipeline.py
│   ├── report.py
│   └── schemas.py
├── tests/
│   ├── integration/
│   │   └── test_live_analysis.py
│   └── unit/
│       ├── test_config.py
│       └── test_report.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

Setup files

`requirements.txt`:

```
langchain-core
langchain-openai
openai
pydantic
python-dotenv
```

`.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. `LICENSE` is the MIT text. Every folder under `src/` gets an empty `__init__.py`.

A sample review batch (`samples/reviews.txt`)

```
# samples/reviews.txt
"The battery life is amazing and the screen is gorgeous." - 5 stars
"Love the screen, but setup took me an hour and the manual was useless." - 3
"Great hardware. Way too expensive for what it is though." - 3 stars
"Battery lasts all day. Setup was confusing but support helped." - 4 stars
```

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
```

```
def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The typed shapes and the compiler

```
# src/schemas.py
# The typed shapes the extractors fill in and the compiler folds together.
# Praises and Complaints wrap a list so with_structured_output has a single
# object to return.
from typing import List

from pydantic import BaseModel

class Praises(BaseModel):
    items: List[str] = []

class Complaints(BaseModel):
    items: List[str] = []

class FeedbackReport(BaseModel):
    summary: str
    praises: List[str] = []
    complaints: List[str] = []
```

```
# src/report.py
# Fold the three extractor results into one FeedbackReport and format it for a
# human. Plain Python, fully testable with hand-made parts - no model.
from src.schemas import FeedbackReport

def compile_report(parts) -> FeedbackReport:
    return FeedbackReport(
        summary=parts["summary"],
        praises=parts["praises"].items,
        complaints=parts["complaints"].items,
    )

def format_report(report: FeedbackReport) -> str:
    lines = ["Overall mood:", "  " + report.summary, "", "Praised:"]
    for praise in report.praises:
        lines.append("  - " + praise)
    lines.append("")
    lines.append("Complaints:")
    for complaint in report.complaints:
        lines.append("  - " + complaint)
    return "\n".join(lines)
```

The three extractor agents

```
# src/agents/praises.py
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import Praises

PRAISES_PROMPT = (
    "From the customer reviews below, list the things customers liked or "
    "praised, short phrases. If none, return an empty list.\n\n"
    "Reviews:\n{reviews}"
)

def build_praises_chain(model):
    prompt = ChatPromptTemplate.from_template(PRAISES_PROMPT)
    return prompt | model.with_structured_output(Praises)
```

```
# src/agents/complaints.py
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import Complaints

COMPLAINTS_PROMPT = (
    "From the customer reviews below, list the complaints or problems "
    "customers raised, short phrases. If none, return an empty list.\n\n"
    "Reviews:\n{reviews}"
)

def build_complaints_chain(model):
    prompt = ChatPromptTemplate.from_template(COMPLAINTS_PROMPT)
    return prompt | model.with_structured_output(Complaints)
```

```
# src/agents/summary.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

SUMMARY_PROMPT = (
    "In 1-2 plain sentences, describe the overall mood across the customer "
    "reviews below.\n\nReviews:\n{reviews}"
)

def build_summary_chain(model):
    prompt = ChatPromptTemplate.from_template(SUMMARY_PROMPT)
    return prompt | model | StrOutputParser()
```

The parallel pipeline

```
# src/pipeline.py
# Run the three extractor chains at the same time with RunnableParallel (they
# don't depend on each other), then fold the results into one FeedbackReport.
from langchain_core.runnables import RunnableParallel

from src.agents.complaints import build_complaints_chain
from src.agents.praises import build_praises_chain
from src.agents.summary import build_summary_chain
from src.report import compile_report
from src.schemas import FeedbackReport
from src.utils.logger import get_logger

def build_pipeline(model):
    return RunnableParallel(
        summary=build_summary_chain(model),
        praises=build_praises_chain(model),
        complaints=build_complaints_chain(model),
    )

def run_analysis(model, reviews: str) -> FeedbackReport:
    logger = get_logger(__name__)
    parts = build_pipeline(model).invoke({"reviews": reviews})
    report = compile_report(parts)
    logger.info(
        f"analyzed: {len(report.praises)} praise(s), "
        f"{len(report.complaints)} complaint(s)"
    )
    return report
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_analysis
from src.report import format_report

def main():
    if len(sys.argv) < 2:
        print("usage: python main.py <reviews-file>")
```

```
        return
    path = sys.argv[1]
    with open(path) as reviews_file:
        reviews = reviews_file.read()

    config = load_config()
    model = get_model(config)
    report = run_analysis(model, reviews)
    print()
    print(format_report(report))

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "FEEDBACKLENS_TEST_SETTING"

def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
```

```
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_report.py
# Run from the repo root: python -m tests.unit.test_report
from src.report import compile_report, format_report
from src.schemas import Complaints, Praises

def sample_parts():
    return {
        "summary": "Mostly positive — great hardware, rough setup.",
        "praises": Praises(items=["Battery life", "Screen quality"]),
        "complaints": Complaints(items=["Setup is confusing",
                                        "Price is too high"]),
    }

def test_report_lists_both() -> None:
    text = format_report(compile_report(sample_parts()))
    result = "OK"
    if "Battery life" not in text:
        result = "WRONG"
    if "Price is too high" not in text:
        result = "WRONG"
    print(f"case 1 (praises + complaints) -> {result}")

def test_summary_is_shown() -> None:
    text = format_report(compile_report(sample_parts()))
    result = "OK"
    if "Mostly positive" not in text:
        result = "WRONG"
    print(f"case 2 (mood summary shown)   -> {result}")

if __name__ == "__main__":
    test_report_lists_both()
    test_summary_is_shown()
```

```
# tests/integration/test_live_analysis.py
# Real model. Run from the repo root:
#   python -m tests.integration.test_live_analysis
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_analysis

REVIEWS = (
    "The battery is amazing but setup was confusing.\n"
```

```
    "Love the screen. Too expensive though."
)

def test_finds_praise_and_complaint() -> None:
    config = load_config()
    model = get_model(config)
    report = run_analysis(model, REVIEWS)
    ok = len(report.praises) >= 1 and len(report.complaints) >= 1
    print(f"case 1 (>= 1 praise + complaint) -> {'OK' if ok else 'WRONG'}")

if __name__ == "__main__":
    test_finds_praise_and_complaint()
```

Expected unit output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (praises + complaints) -> OK
case 2 (mood summary shown)   -> OK
```

The whole project, in one sentence

Three specialist chains — praises, complaints, a mood summary — run at the same time with `RunnableParallel` and compiled by deterministic code into one report, so a pile of reviews becomes clean, structured signal. The same patterns as MeetingMind, on an entirely different kind of text.